AOSP 进阶丛书 · 第一册 · ANDROID 13 (API 33)

Android 13 全栈架构总览

基于 AOSP Android 13 (Tiramisu) 完整源码的全景导读：五层架构、源码树地图、HAL 三代演进、构建与分区体系、ART 运行时、SELinux 与属性系统——每个论断附真实源码路径与行号。

源码基线：AOSP Android 13 (TP1A · SDK 33) · 行号全部实测核验

目 录

前言：从应用开发者到系统开发者 2
第一章 五层架构总论 2
第二章 源码树全景导览 4
第三章 内核层与 HAL 层 6
第四章 Native 世界：守护进程与基础库 7
第五章 Framework 层：API、服务与分发中枢 8
第六章 层间通信机制总览 10
第七章 构建与产物体系 11
第八章 运行时体系：ART 与 Zygote 13
第十章 深入属性系统：一次 setprop 的完整旅程 14
第十一章 SELinux：系统的第二套权限体系 15
第十二章 图形与输入骨架概览 17
第十三章 观测与调试：把系统打开给你看 17
第十四章 进阶路线图：本丛书之后怎么走 19
第十五章 设计决策分析（全书总结） 19
第十六章 深水区专题一：SELinux 策略实战与 APEX 内部 20
第十七章 深水区专题二：Soong 实战与链接器命名空间 21
第十八章 边境机制三讲：JNI、Context 家族与 native 智能指针 22
附录 A 版本指纹与阅读路线 23
附录 B 系统进程归属速查 24
附录 C 常用系统属性速查（带源码出处） 24
附录 D 术语表 25
附录 E 上手实验（半天量） 26

本册速览

五层架构与贯穿机制（Binder/属性/SELinux/APEX）——任何模块 30 秒定位
源码树地图 · 分区与构建（Soong/APEX/AVB） · HAL 三代演进 · ART 三级执行
深水区：SELinux 策略实战 · apexd 回滚 · Soong 写模块 · 链接器命名空间
边境三讲：JNI 三类引用 · Context 家族 · sp/wp/RefBase
建议顺序：通读建框架 → 对照锚点精读 → 附录 E 实验 → 进第二册

前言：从应用开发者到系统开发者

本书是《AOSP 进阶丛书》的第一册，基于你本机的 **AOSP Android 13（Tiramisu，API 33，安全补丁 2022-08-05）** 完整源码树撰写，所有结论都给出了 **文件:行号** 级别的源码锚点，可在 `/run/media/marcusgu/SoftWare/aosp` 下逐一验证。

作为应用开发者，你已经熟悉 SDK 的世界：Activity、Service、Handler、Gradle。而系统开发者的世界从这里开始向下延伸——你的代码之下的 `framework.jar` 是怎么运行起来的，`getSystemService` 拿到的到底是什么，开机 logo 之后到 Launcher 出现之前发生了什么。本丛书沿着“**开机 → 系统就绪 → 应用启动 → 进程间通信**”这条贯穿全栈的主线组织内容：

- **第一册（本书）**：全景地图。五层架构、源码树导览、构建与分区体系、运行时体系。读完你将获得“任何模块都能定位到它属于哪一层、住在哪个目录、依赖谁”的能力。
- **第二册**：系统启动主线。init → Zygote → SystemServer 的逐函数精读。
- **第三册**：Binder IPC 全链路。从内核驱动到 Java 层的完整调用链剖析。

阅读方法建议：先通读建立框架，再对照源码锚点精读；每册末尾的“设计决策”一章是面试与架构能力的分水岭，值得反复咀嚼。

> **设计洞察 · 本丛书的源码约定**
> 本书所有行号基于 Android 13（TP1A）源码树实测；目录路径均相对源码树根 `$A`。源码树严格只读——研读系统代码的首要纪律是：先理解，再动手。

第一章 五层架构总论

Android 的软件栈通常被划分为五层。分层的意义不在于“好看”，而在于**依赖方向的确定性**：每一层只依赖它下面的一层，越往上越贴近应用，越往下越贴近硬件。

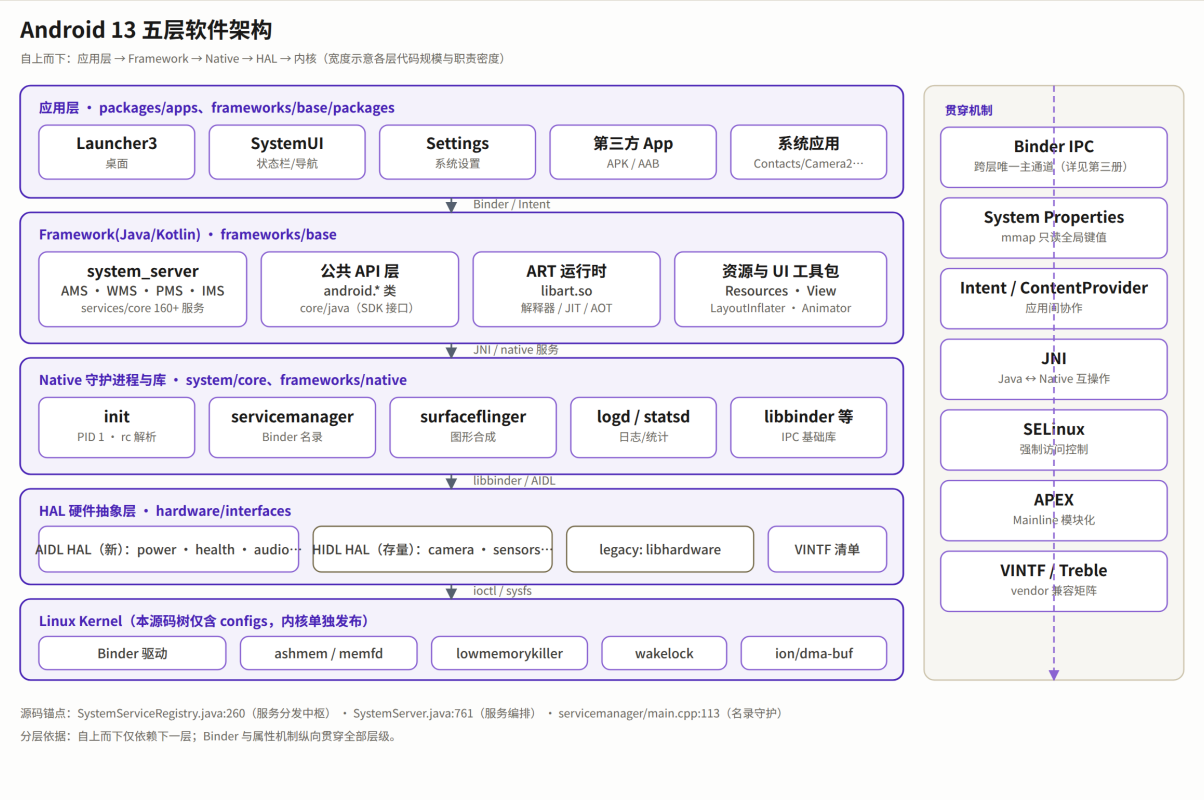


图 1 Android 13 五层软件架构总图：左侧自上而下为应用层、Framework、Native、HAL、内核，右侧为纵向贯穿的公共机制

1.1 自上而下看五层

应用层运行在独立的 App 进程中，通过 SDK API 与系统交互。系统应用大多住在源码树两处：`packages/apps/`（Launcher3、Settings、Camera2 等）与 `frameworks/base/packages/SystemUI`。它们与第三方 App 本质相同——都靠 Binder 调用系统服务，只是签名与权限不同。

Framework 层是系统开发者的主战场，物理上由 `frameworks/base` 承载。它包含两部分：公共 API（`core/java/android/`，即编译进 `framework.jar` 的 `android.*` 类）与系统服务实现（`services/core/java/com/android/server/`，编译进 `services.jar`，运行在 system_server 进程）。你调用的每个系统 API，最终几乎都穿过前者、经 Binder 落到后者。

Native 层是 C/C++ 的世界：`system/core` 与 `frameworks/native` 中的守护进程（init、servicemanager、surfaceflinger、logd 等）和基础库（libbinder、libutils、libcutils）。渲染合成、输入分发这类对性能极敏感的职责都在这一层完成。

HAL（硬件抽象层）把“芯片相关”与“系统无关”隔离开：接口定义集中在 `hardware/interfaces/`，实现由各 SoC 厂商提供在 vendor 分区。Android 13 的主线是 AIDL HAL，存量 HIDL 仍大量并存。

内核层是 Linux kernel 加上 Android 特有补丁（Binder 驱动、ashmem/memfd、lowmemorykiller、wakelock 等）。值得注意的是：**本源码树的 `kernel/` 目录只有 configs、prebuilts、tests 三个子目录，不含内核源码**——AOSP 的内核是独立仓库（如 android13-5.10），

但内核的稳定 UAPI 头（包括 Binder 协议的 `binder.h`）通过 `external/kernel-headers/original/uapi/` 进入编译。

1.2 右侧“贯穿列”：真正的系统观

初学者容易把五层理解成“五块积木”，但真正的架构能力在于看见**纵向贯穿的机制**：

- **Binder IPC** 几乎是所有跨层调用的唯一通道：App → system_server 是 Binder，system_server → HAL 服务还是 Binder。第三册将逐跳剖析它。
- **System Properties** 是全机共享的键值区：init 建立，所有进程 mmap 只读共享，`persist.` 前缀落盘。它是比 Binder 轻得多的“单向广播”。
- **SELinux** 的策略覆盖从 init 第一个动作到每个服务的每一次访问——`system/sepolicy/` 是策略源码，理解它才能理解“为什么这个服务连不上那个端口”。

1.3 实战：一行代码看穿五层

以应用里最常见的调用为例——`powerManager.isInteractive()`（判断屏幕是否点亮）：

java 应用进程内（五层穿越起点）

```
PowerManager pm = context.getSystemService(PowerManager.class);
boolean lit = pm.isInteractive();
```

它的下行路径是：`PowerManager`（应用层 API，`core/java/android/os/PowerManager.java`）持有一个 `IPowerManager` Binder 代理 → `SystemServiceRegistry` 在首次获取时经 `ServiceManager.getService("power")` 从 **servicemanager**（Native 层）查到 system_server 注册的电源服务 Binder 句柄 → 调用发起后经 **Binder 驱动**（内核层）进入 system_server（Framework 服务层）的 `PowerManagerService.isInteractive()`（`services/core/.../power/PowerManagerService.java`）→ 返回值原路回到应用。一行代码，五层全穿。**本丛书剩余两册，就是把这条线上的每一层拆开讲透。**

> **设计洞察 · 分层的本质是依赖管理**
>
> 五层中任何一层被替换，只要接口契约不变，上下层不受影响。这正是 Treble 工程的落点：把 vendor 相关的 HAL/内核钉死接口，让 system 分区可以独立升级。判断一个改动“该放在哪一层”的系统思维：能否被多个应用复用 → Framework；是否性能关键且稳定 → Native；是否因硬件而异 → HAL。

第二章 源码树全景导览

面对数 GB 的源码树，系统开发者的第一项能力是“看目录知职责”。本章给出一张地图，并标出本丛书的精读路径。



图 2 AOSP 源码树地图：上半部分为核心目录分组，下半部分为“从开机到 App”主线的落点

2.1 必须烂熟于心的五个目录

目录	职责	关键子目录
`frameworks/base`	Framework 全部	core/java（API）、services（服务端）、cmds/app_process（Zygote）
`system/core`	基础组件	init、rootdir（init.rc）、debuggerd、libutils
`frameworks/native`	图形与 IPC	libs/binder（libbinder）、cmds/servicemanager、services/surfaceflinger
`hardware/interfaces`	HAL 接口定义	power、audio、health 等 AIDL/HIDL 接口与 VINTF 矩阵
`art`	运行时	runtime（虚拟机）、dex2oat（AOT 编译器）、compiler

其余目录按需检索：`packages/`（系统应用）、`bionic`（Android 版 libc）、`libcore`（Java 核心库，含 OpenJDK 移植 `ojluni`）、`build/`（构建系统）、`external/`（数百个第三方项目）、`prebuilts/`（预置工具链）。

2.2 用目录回答三个问题

“servicemanager 在哪？” 它在 `frameworks/native/cmds/servicemanager/`（本树位置；较老教材写 `system/core/servicemanager`，Android 13 已迁移）。这提醒我们：**教材会过时，目录检索**

能力不会。

“init.rc 有几份？” 入口是 `system/core/rootdir/init.rc`，它 import 了 `init.${ro.hardware}.rc`、`init.${ro.zygote}.rc` 等；每个守护进程还各自带 `/system/etc/init/*.rc`。第二册将精读这条 import 链。

“我该改哪层的代码？” 先看现象的层级：API 行为问题查 `frameworks/base/core/java` 与 `services/core`；性能/图形查 `frameworks/native`；设备差异查 `hardware/interfaces` 与 `device/`。

> **源码锚点 · 快速检索工具链**
> ripgrep 全树检索：rg -t cpp "SecondStageMain" --line-number
按文件名定位：find . -name "ZygoteInit.java" -not -path "*/out/*"
确认版本：build/make/core/version_defaults.mk（SDK_INT 33）
排除产物：始终加 -not -path "*/out/*" 避免命中构建产物

第三章 内核层与 HAL 层

3.1 Android 对 Linux 内核的增强

Android 内核 = 标准 Linux + 一组"Android 通用内核"补丁。最重要的五项：

- **Binder 驻动**（`drivers/android/binder.c`，不在本树）：进程间通信的基石，一次拷贝与内核级身份校验都由它实现（第三册详述）。UAPI 定义在 `external/kernel-headers/original/uapi/linux/android/binder.h`。
- **ashmem/memfd**：匿名共享内存，Binder 传递大块数据（如位图）的配套机制。
- **lowmemorykiller 演进**：内核态 LMK 已被用户态 `lmkd`（`system/core/lmkd`）+ PSI 压力信息取代，通过 socket 与 init 体系协同。
- **wakelock/wakeup source**：电源管理的同步原语，`libsuspend`（`system/core/libsuspend`）封装。
- **ION/DMA-BUF**：显存缓冲分配，现已被内核标准 DMA-BUF 取代。

3.2 HAL：三代演进

HAL 的历史是 Android 对“碎片化”宣战的历史。legacy HAL 把厂商代码编进 Framework 进程，升级系统必须重编厂商代码；Android 8 的 Treble 用 HIDL 强制进程隔离与版本化；Android 10 起 AIDL HAL 统一语言栈，让 HAL 与系统服务共用同一套 Binder 设施与工具链。



图 3 HAL 三代演进时间轴与 AIDL HAL 调用路径、VINTF 兼容矩阵

> **设计洞察 · 为什么 HIDL 被 AIDL 取代**
>
> HIDL 达成了隔离目标，但代价是第二套 IDL 编译器、第二个 binder 域（/dev/hwbinder）、第二个服务管理器（hwservicemanager）。AIDL HAL 复用主线设施后，工具链、调试手段、线程模型全部归一——工程上“少一套系统”的收益压倒了迁移成本。存量 HIDL（如部分 camera 接口）长期冻结维护，读代码时先看 `aidl/` 还是 `1.x/` 目录区分代际。

第四章 Native 世界：守护进程与基础库

Native 层的守护进程几乎全部由 init 在启动各阶段拉起（谁在什么阶段启动，第二册的触发链图给出完整答案）。本册先建立名录。

4.1 核心守护进程名录

进程	源码位置	职责	启动类别
`init`	system/core/init	PID 1，解析 rc、孵化与监护所有服务	内核第一个用户态进程
`servicemanager`	frameworks/native/cmds/servicemanager	Binder 服务名录（句柄 0）	class core
`surfaceflinger`	frameworks/native/services/surfaceflinger	图形图层合成、VSync 分发	class core
`logd`	system/logging/logd	全机日志汇聚	class core
`lmkd`	system/memory/lmkd	低内存查杀	class core
`vold`	system/vold	卷管理、加密、fuse	early-fs 阶段
`netd`	system/netd	网络策略与 DNS	class main

进程	源码位置	职责	启动类别
audioserver / cameraserver / mediaserver	frameworks/av 等	媒体相关服务进程	class main
bootanimation	frameworks/base/cmds/bootanimation	开机动画（开机完成的视觉信号）	class core animation

“类别”（class core/main/hal…）不是注释，而是 init 批量启动的调度单位——class_start main 一条命令拉起整组服务，详见第二册 builtins.cpp 的 do_class_start。

4.2 基础库版图

- **libbinder**（frameworks/native/libs/binder）：Native 侧 Binder 运行库，ProcessState/IPCThreadState 所在地，第三册主角。
- **libutils**（system/core/libutils）：RefBase、sp/wp 智能指针、Looper、Thread——Native 世界的“公共底座”。
- **libcutils**（system/core/libcutils）：属性读写、socket 工具、trace 等 C 工具集。
- **libgui / libui**（frameworks/native/libs/{gui,ui}）：BufferQueue、Surface、图形缓冲——渲染栈的 Native 骨架。

> **源码锚点 · 从守护进程到“服务类”的对应**
> servicemanager.rc：class core animation，critical —— 名录自身也是被 init 管理的服务
init.zygote64.rc：class main —— Framework 全部进程的祖先进“主类”
SystemServer.java:435：sMaxBinderThreads=31 —— system_server 的 Binder 线程预算

4.3 启动性能观测的一件老兵器：bootchart

system/core/init/bootchart.cpp 是 init 内建的启动时间线采集器：设置 persist.init.bootchart 后，init 记录各 action/服务时间戳，配合 system/core/init/grabbootchart.sh 产出甘特图。它是第二册“触发链”一章的天然实验伙伴——改一个 rc 触发顺序，bootchart 立刻告诉你启动时间花在哪。Google 现代工作流更多用 perfetto（external/perfetto），但 bootchart 与 init 代码同源同寿命，读懂它等于复习 init 的数据结构。

第五章 Framework 层：API、服务与分发中枢

5.1 一份代码，两个 jar

Framework 的物理形态是两个部署单元：framework.jar（所有进程共享的 API 与基础类）与 services.jar（只在 system_server 进程加载的服务实现）。App 进程通过 Zygote 的写时复制共享

framework.jar 的内存映射——这是全机内存优化的大头（第二册 Zygote 章详述）。

API 与实现的边界藏在 frameworks/base 的目录结构里：core/java/android/app/ActivityManager.java 是给应用调用的门面，真正逻辑在 services/core/java/com/android/server/am/ActivityManagerService.java（1.8 万行）。门面通过 Binder 把请求送进 system_server。

5.2 getSystemService 的分发中枢

应用里再平常不过的一行 context.getSystemService(Context.POWER_SERVICE)，背后是 frameworks/base/core/java/android/app/SystemServiceRegistry.java 这个中枢：

> **源码锚点 · SystemServiceRegistry：服务注册表**
> SystemServiceRegistry.java:260 类定义 —— 静态注册 100+ 服务的 fetcher
SystemServiceRegistry.java:270 SYSTEM_SERVICE_FETCHERS 全局 map
SystemServiceRegistry.java:284 CachedServiceFetcher：进程级缓存的管理器对象
SystemServiceRegistry.java:289+ 各服务以 registerServiceFetcher 注册获取逻辑

多数 fetcher 的逻辑是：拿 Context → 通过 ServiceManager.getService("power") 拿到 Binder 代理 → IPowerManager.Stub.asInterface 包装 → 缓存。也就是说，**管理器对象只是 Binder 代理的本地缓存马甲**。看懂这一点，你就打通了“应用层 API → Binder → system_server”的静态地图；动态的每一步在第三册。

5.3 system_server 里的服务版图

system_server 以单进程容纳 160+ 系统服务（第二册给出启动时序），按包名归类：am/（AMS 及进程/广播/服务管理）、wm/（WMS 与窗口/ATM）、pm/（PMS 包管理）、power/、input/、display/、notification/、usb/……而图形合成（SurfaceFlinger）、音频（audioserver）等因性能与崩溃隔离**有意留在外面**——集中与隔离的权衡，是第十章设计决策的主题之一。

5.4 常见系统服务速查表

getSystemService 返回	Binder 名	实现类（services/core/...）	所在进程
ActivityManager	activity	am/ActivityManagerService	system_server
WindowManager	window	wm/WindowManagerService	system_server
PackageManager	package	pm/PackageManagerService	system_server
PowerManager	power	power/PowerManagerService	system_server
InputManager	input	input/InputManagerService	system_server
DisplayManager	display	display/DisplayManagerService	system_server

getSystemService 返回	Binder 名	实现类（services/core/...）	所在进程
NotificationManager	notification	notification/NotificationManagerService	system_server
LocationManager	location	location/LocationManagerService	system_server
AudioManager	audio	（跨进程）audio 服务	audioserver
CameraManager	camera	（跨进程）camera 服务	cameraserver
SensorManager	sensors	（跨进程）sensorservice	sensorserver
WifiManager	wifiservice	wifi/WifiService	system_server

注意“所在进程”一列：多数框架服务集中在 system_server，但音频/相机/传感器有意独立进程——再次呼应“故障域”划分。第三册会以 `activity` 服务为例，把“Binder 名 → 代理对象 → 具体方法”的运行期解析全程展开。

第六章 层间通信机制总览

系统的“血管系统”由四条通道构成，各自解决不同问题：

机制	典型用途	数据形态	成本量级
Binder IPC	一切跨进程方法调用	结构化事务（Parcel）	微秒级，一次拷贝
System Properties	全局状态广播（开机完成、USB 配置）	键值字符串（mmap 共享读）	纳秒级读
Intent/ContentProvider	应用间协作与数据共享	语义化消息/表格数据	Binder 之上封装
JNI	Java ↔ Native 边界	函数调用	纳秒级（经 ART 精心优化）

属性系统值得单说：init 在第二阶段调用 `PropertyInit()`（`init.cpp:887`）建立 `/dev/__properties__` 共享区，所有进程从此 **mmap 只读**读取；写入则经 socket 连到 init 的属性服务（`property_service.cpp:527` 的 `handle_property_set_fd`），由 init 统一裁决（含 SELinux 检查）后广播。**读路径零系统调用、写路径集中仲裁**——这个不对称设计以极低成本支撑了全机数千项配置。

6.1 一次 `SystemProperties.set` 的完整下行链

把上一节的机制落到代码，从应用层下行到 init 的每一跳：

text 属性写入全链（自上而下）

```
应用: SystemProperties.set("debug.hwui.renderer", "skiagl")
↓ (frameworks/base/core/java/android/os/SystemProperties.java, native_set)
JNI: android_os_SystemProperties_set (core/jni/android_os_SystemProperties.cpp)
↓ __system_property_set
bionic: 发送 socket 到 /dev/socket/property_service (libc/bionic/system_properties.cpp)
↓ init 侧 accept4
init: handle_property_set_fd (property_service.cpp:527)
```

```
↓ 凭证 SO_PEERCRED + SELinux 上下文
仲裁：HandlePropertySet(:481) → PropertySet(:176) → 更新共享区 + 持久化(persist.)
↓ 属性变更通知
消费方：poll 属性 fd 或由 init 触发 on property:xxx=* 动作
```

对照第三册即将展开的 Binder 链，你会发现惊人相似的结构：**Java 门面 → JNI → Native 库 → 内核/守护进程仲裁 → 广播反馈**。Android 的“跨边界调用”几乎都是这个形状，差别只在仲裁者是谁（init 属性服务、Binder 驱动或 servicemanager）。

6.2 运行时资源覆盖（RRO）：不改代码的定制面

分区里的 overlay APK（书6 §9 扫描的 :320）在运行时由 **OverlayManagerService**（`services/core/.../om/OverlayManagerService.java`）挂载：把覆盖包的资源表叠加进目标包的 AssetManager——**按资源 ID 重定向**，同一 ID 优先取覆盖值。三层用途递进：厂商暗色主题/默认值定制（vendor overlay）、运行时主题切换（framework 的 android 覆盖）、用户个性化（ThemePicker）。它和 APEX 是一对：APEX 换"代码与库"，RRO 换"资源与配置"——都是"不动 system 分区本体"的定制哲学。读码入口：OMS 的 `setOverlayEnabled` 与 `OverlayActorEnumerator` 策略校验（谁能覆盖谁，受 targetName/签名约束）。

> **设计洞察 · 为什么属性不用 Binder**
> 属性读取发生在进程启动最早期（连 ART 都未就绪），也发生在 SELinux 策略加载前，必须是一个“dumb 到不能再 dumb”的机制。mmap 只读 + init 单点写，正是不依赖任何运行时的答案。

第七章 构建与产物体系

7.1 分区布局

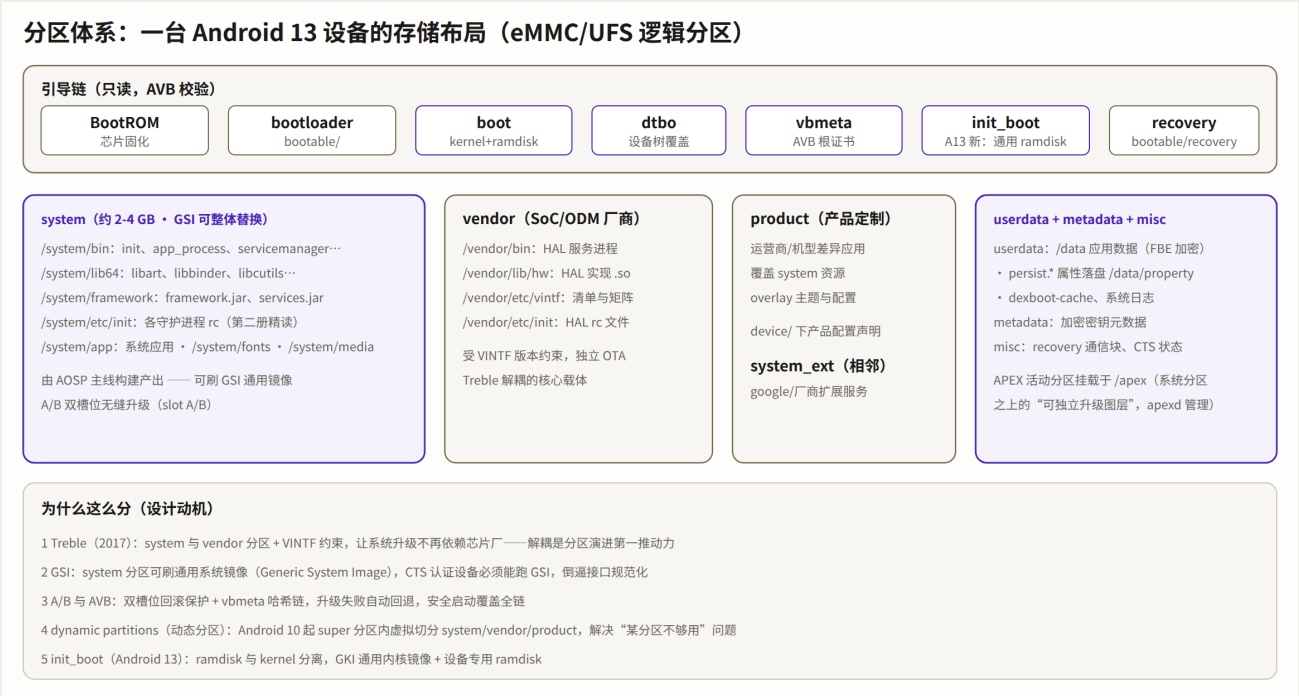


图 4 Android 设备分区布局：引导链、四大主分区与 data 系分区，附五大设计动机

分区的关键词是**解耦与可替换**：system 可刷 GSI 通用镜像，vendor 受 VINTF 接口矩阵约束独立升级，product 承载机型差异。Android 13 还引入 init_boot 分区，把通用 ramdisk 与内核分离，配合 GKI（通用内核镜像）。

7.2 构建流水线



图 5 Soong/Kati→ninja→镜像的构建流水线与 APEX 生命周期、Soong 取代 Make 的动机

系统开发者的日常之一是读 `Android.bp`：`cc_binary` 产出守护进程，`aidl_interface` 从 .aidl 生成 Java/C++ 双侧绑定代码，apex 定义 Mainline 模块。**看一个模块的 Android.bp，就知道它编译成什么、装进哪个分区、暴露哪些接口**——这是改代码前的必做功课。

源码锚点 · 构建体系锚点

build/soong/android：Soong 模块框架（Go）
build/make/core/Makefile：system.img 等镜像组装规则（Kati 侧）
system/apex/apexd：APEX 运行时守护（挂载 /apex）
bionic/apex/com.android.runtime.*：runtime APEX 的 AVB 公钥与证书

第八章 运行时体系：ART 与 Zygote



图 6 ART 三级执行策略（解释器/JIT/AOT + profile 反哺）与 zygote 进程模型、Dalvik→ART 演进动机

应用开发者视角的“Java”，在系统层是三件事：**dex 字节码规范、ART 虚拟机、Zygote 进程模型**。ART 的三级执行（解释器 → JIT → profile 引导的 AOT）让“安装快、启动快、运行快”这三个矛盾目标动态平衡。而 Zygote 把 `framework.jar` 与资源预加载进一个模板进程，之后所有 App fork 而出、共享只读页——第二册将从 `app_main.cpp` 开始走完这条链。

8.1 dex2oat 与启动镜像 boot.art

AOT 的输入输出由 `art/dex2oat` 定义：读入 .dex（vdex 中缓存原文）与 profile，产出 .odex（方法机器码）与 boot 镜像。**boot classpath**（framework.jar 等系统 jar 集）会额外编译为 `boot.art/boot.oat`——它们正是 Zygote 预加载时 mmap 的“共享机器码层”：所有 App 进程共享同一份框架类机器码映射，写时复制只在应用真正修改对象时发生。安装期与空闲期的编译策略由 `art/dex2oat/dex2oat.cc` 与 `system/serverconfiguration` 等协同控制；Cloud Profiles（Android 12+）让云端聚合的热点信息反哺本地 AOT。

源码锚点 · ART 关键产物与位置

art/dex2oat：AOT 编译器（读 dex+profile，产 odex）
/data/dalvik-cache：运行期编译产物目录
boot.art / boot.oat：框架类共享机器码（zygote 映射）
ZygoteInit.java:247 preloadClasses：预加载类清单的消费者（第二册）

设计洞察 · 为什么 App 进程必须从 Zygote fork

若每个进程自行加载 200MB 级的类与资源，启动要数秒且内存爆炸。fork + 写时复制让“加载”只发生一次；代价是所有 App 共享同一个 ART 运行时版本——这也是 Mainline 化 runtime APEX 的原因之一：运行时可独立升级，但升级必须整机重启生效。

第十章 深入属性系统：一次 setprop 的完整旅程

属性系统是“最小的系统机制”，却五脏俱全——值得作为第一个逐函数精读的对象。

10.1 属性的三个存储域

- **默认域**：开机时 init 从 `/system/etc/prop.default`、`/vendor/build.prop`、`/data/property/persistent_properties` 等加载，见 `property_service.cpp:639` 的 `LoadProperties`。
- **共享内存区**：init 创建 `/dev/__properties__`，各域按前缀分区；所有进程 mmap **只读**。
- **persist 域**：`persist.` 前缀的属性由 `persistent_properties.cpp` 序列化到 `/data/property/persistent_properties`，重启不丢。

10.2 写入路径精读

任何进程调用 `SystemProperties.set`（bionic 的 `__system_property_set`）后发生的事：

cpp system/core/init/property_service.cpp:176（节选）

```
static uint32_t PropertySet(const std::string& name, const std::string& value,
std::string* error) {
size_t valuelen = value.size();
if (!IsLegalPropertyName(name)) { ... } // 1. 合法性：长度/字符集
if (valuelen >= PROP_VALUE_MAX) { ... }
if (StartsWith(name, "ctl.")) { // 2. ctl.* 是控制命令不是属性
return HandleControlMessage(name.c_str() + 4, value, getuid());
}
...
if (persistent_properties_loaded && StartsWith(name, "persist.")) {
error->insert(0, "Persistent property not loaded yet"); // 3. persist 未就绪前拒绝写
}
...
}
```

设计洞察 · 逐段精读：这段 30 行代码里的四道闸

①合法性（名字字符集/长度、值长度）——防注入与滥用；②`ctl.` 前缀劫持——属性写接口兼任服务控制通道（setprop ctl.start zygote 即可拉服务），但随后立即校验调用者权限；③persist 就绪检查——/data 未挂好前拒绝持久化，防止"丢在内存里装作持久化"；④`ro.` 只读——一次写后封死。四道闸合起来就是"一个最小 API 的完整安全叙事"。

三个细节值得一读：`ctl.` 前缀被劫持为**控制通道**（`ctl.start/ctl.stop` 直接驱动服务启停，等价于 `setprop ctl.start zygote`）；写入者身份（`getuid`）被记录；`ro.` 前缀只读属性在设置一次后返回 `PREADONLY`。

仲裁入口 `HandlePropertySet`（`:481`）先做 **SELinux source_context 检查**——不是任何进程都能写任何属性；通过 socket 收包的部分在 `handle_property_set_fd`（`:527`），它 accept 连接、读属性名与值、取对端凭证（`SO_PEERCRED`），再进入仲裁。**凭证来自内核，不可伪造**——这个模式 Binder 里将再次出现（第三册 `tr.sender_euid`）。

设计洞察 · 面试题常客：属性能跨 SELinux 域随便写吗

不能。属性按前缀划分 SELinux 上下文（property_contexts 文件），写 `sys.powerctl` 需要对应域授权。这也是为什么“重启设备”属性普通 App 写不了——策略层早已拒绝。

10.3 读取路径：为什么读不要系统调用

bionic 的 `__system_property_find` 直接在 mmap 的属性区里按前缀定位域、二分查找。属性区布局与查找算法在 `bionic/libc/bionic/system_property_api.cpp` 与 `system/core/property_service/libpropertyinfoparser` 中——读路径没有任何内核参与，这就是“纳秒级读”的来源。应用侧的 `SystemProperties.java` 只是 JNI 薄封装。

源码锚点 · 属性系统锚点速查

init.cpp:887 PropertyInit：第二阶段建立属性区
init.cpp:912 StartPropertyService：打开 /dev/socket/property_service
property_service.cpp:176 PropertySet：写仲裁（合法性/ctl./persist）
property_service.cpp:527 handle_property_set_fd：socket 收包与凭证
SystemServer 启动后 sys.boot_completed=1（AMS.java:5149，第二册详述）

第十一章 SELinux：系统的第二套权限体系

传统 Linux 权限（DAC）回答“你是谁、文件给谁读写”；SELinux（MAC）回答“**你的域允许对什么类型做什么**”。Android 从 5.0 全量强制（enforcing），策略源码就在 `system/sepolicy/`。

11.1 从 init 的第一个动作看起

第二阶段 init 在属性初始化后立即做三件事（init.cpp:901-903）：SelinuxSetupKernelLogging、SelabelInitialize（加载 file_contexts）、SelinuxRestoreContext（selinux.cpp:762，按标签规则递归恢复全文件系统标签）。**在第一个服务启动前，规则就已就位**——此后每个服务进程 exec 时（service.cpp 的 SetExecContext）都会切入自己的 SELinux 域。

11.2 策略文件的分工

文件/目录	作用
system/sepolicy/public	导出给 vendor 使用的类型与接口（system_ext 同理）
system/sepolicy/private	system 私有策略
system/sepolicy/vendor	vendor 侧策略默认集
*.te 文件	域规则：allow/init neverallow，如 allow init zygote_exec:file { read execute }
file_contexts	路径 → 文件类型标签（/system/bin/app_process64 → zygote_exec）
service_contexts	Binder 服务名 → 服务标签（activity → activity_service）
property_contexts	属性前缀 → 属性标签

读懂三张 contexts 表，是排查“服务起不来/属性写不进/设备节点打不开”类问题的钥匙。而 neverallow 规则是策略的“宪法条款”——编译期强制，谁也不能悄悄放水。

一个最小的服务域定义长什么样（示意，风格取自 system/sepolicy）：

text 服务域示意（.te 片段）

```
type mysvc, domain; # 声明域类型
type mysvc_exec, exec_type, file_type; # 可执行文件类型
init_daemon_domain(mysvc) # 由 init 拉起并切入域
allow mysvc activity_service:service_manager find; # 允许查询 activity 服务
neverallow mysvc shell_data_file:file { read write }; # 永久禁止碰 shell 数据
```

读 avc denied 日志的方法：avc: denied { find } for comm="mysvc" scontext=u:r:mysvc:s0 tcontext=u:object_r:activity_service:s0——按 scontext/tcontext 反查缺少的 allow 行，把最小权限补进策略。

> **设计洞察 · 系统开发者与 SELinux 的日常**
> 新增系统服务必须：定义 .te 域 + service_contexts 注册 + rc 文件声明 selinux 域。忘了会看到 avc denied 日志（logcat -b events | grep avc）。调试期可 setenforce 0 临时全局放开（仅 userdebug），但**任何 denied 都应通过策略显式 allow 解决**，而不是永久关闭。

第十二章 图形与输入骨架概览

渲染与输入是“系统开发者进阶深水区”，本册只画骨架坐标，细节留给后续批次。

图形一侧的三个角色：App 进程（`frameworks/base` 的 View 系统 + `libhwui`）把界面渲染进 BufferQueue 的图形缓冲；`system_server` 的 WMS 管理窗口层级与生命周期；`surfaceflinger`（`frameworks/native/services/surfaceflinger`）按 VSync 节拍把各窗口缓冲合成送显。三者之间传递的“货币”是 BufferQueue 里的 GraphicBuffer（DMA-BUF），控制流走 Binder，数据流走共享内存——**控制与数据分离**是图形栈的核心设计。

输入一侧：`InputReader/InputDispatcher`（`frameworks/native/services/inputflinger`）从内核 evdev 读取事件，经 `system_server` 的 `InputManagerService` 与 WMS 的窗口焦点信息，把事件注入焦点窗口的 socket。App 侧 `InputEventReceiver` 收到后再分发给 View 树。

VSync 是整个图形栈的心跳。SurfaceFlinger 从 HWComposer 获得硬件 VSync，软件分发（DispSync/VsyncSchedule）给两路客户：App 侧的 Choreographer（`frameworks/base/core/java/android/view/Choreographer.java`，驱动 doFrame 绘制节奏）与 SurfaceFlinger 自身的合成节拍。“App 按节拍生产、SF 按节拍消费”的三缓冲流水线，是 60/120Hz 流畅的基础；渲染专册将沿 VSync 一路深挖。

> **源码锚点 · 图形/输入锚点**
> frameworks/native/services/surfaceflinger/SurfaceFlinger.cpp：合成主循环
frameworks/native/services/inputflinger/reader/InputReader.cpp：事件读取
SystemServer.java:1570：InputManagerService 创建（startOtherServices 阶段）
SystemServer.java:1586：WindowManagerService.main 启动点

第十三章 观测与调试：把系统打开给你看

系统开发者的另一半功夫是**观测**。所有工具的原理都在源码里：

13.1 四件套

工具	看什么	源码原理
`adb shell service list`	已注册 Binder 服务	向 servicemanager（句柄 0）枚举 mNameToService
adb shell dumpsys <服务>	服务内部状态	调用服务的 dump() 方法——每个服务类自己实现（如 ActivityManagerService.dump）
`logcat`	全机日志	logd 汇聚 /dev/log/*，logcat 为客户端

工具	看什么	源码原理
perfetto / systrace	时间线与卡顿	atrace 埋点（ATRACE_CALL 宏遍布本丛书精读过的启动代码）

13.2 启动观测的具体落点

本丛书第二册的每个环节都有配套观测：dmesg 看 init 两阶段日志（"init second stage started!"，init.cpp:839）；logcat -b events 的 boot_progress_* 事件（如 boot_progress_preload_start，ZygoteInit.java:887）；dumpsys activity services、dumpsys window 验证服务与窗口状态；sys.boot_completed 属性作为“开机完成”的唯一权威信号（ActivityManagerService.java:5149 设置）。**先有观测，再谈优化**——这是系统性能工作的第一原则。

13.3 编译与实验环境

本树为完整可构建树。典型工作流：source build/envsetup.sh → lunch（选择目标，如 aosp_x86_64-userdebug）→ m（Soong 构建镜像）。虚拟设备 Cuttlefish（device/google/cuttlefish）可免真机做系统实验。日常单模块迭代用 m services（只编 services.jar）等目标 + adb root; adb sync。**本丛书只读研读，不改源码树**；但读者可以在自己的分支上练习。

dumpsys 的输出就是服务的自我陈述，例如：

text adb shell dumpsys power（节选）

```
POWER STATE (dumpsys power)
mWakefulness=Awake # 唤醒状态机当前值
mUserActivityTimeoutOverrideFromWindowManager=-1
mWakeLockSummary=0x1a # 活跃 wakelock 汇总
mUserActivitySummary=0x1
mHoldDisplaySuspendBlocker=true
mDisplay=Ready # 显示就绪标志
```

每个字段的赋值点都能在 PowerManagerService.java 中 grep 到——**dumpsys 输出即源码索引**，这是系统调试的高效路径：先 dumpsys 定位异常字段，再回源码找字段的所有写点。

（第十三章末的自检问题若都能回答，可径直进入第二册。）

> **设计洞察 · 自检问题（试试不回看正文回答）**
> 1) 五层各自只依赖哪一层？2) servicemanager 在哪个目录，为什么它不需要注册就能被找到？3) system.img 与 vendor.img 的接口契约由什么文件保证？4) 一个属性从 set 到生效经过哪些裁决？5) 为什么 App 进程必须是 fork 的产物？——答不上来的小节值得重读。

第十四章 进阶路线图：本丛书之后怎么走



图 7 从应用开发者到系统开发者的三阶段路线、配套技能栈与自测里程碑

路线图给了三阶段与里程碑，这里补三句话原则：**第一，主线优先于专题**——先走完启动与 Binder 两条主线，再挑专题深潜，否则专题知识挂不住。**第二，输出倒逼输入**——每读完一册，合上书画一遍时序图/调用链，画不出的地方就是没懂的地方。**第三，让源码树成为日常**——把 riagrep 检索、dumpsys 验证变成肌肉记忆，系统开发者的核心竞争力不是背下架构，而是“面对陌生问题时的定位速度”。

丛书后续批次（AMS 与四大组件、WMS 与渲染栈、PMS 与安装流程）将沿用本册的写法：真实源码锚点 + 执行链路 + 设计决策。

第十五章 设计决策分析（全书总结）

本章把散落各章的“为什么”收拢成十条可复用的架构判断：

- **分层与依赖方向**：每层只依赖下一层，跨层只能走公共通道（Binder/属性）。反过来“哪层都不属于”的代码，就是放错位置的代码。
- **Binder 单一 IPC 抽象**：统一的安全模型（内核验 UID/PID）胜过多个“更快”但各搞一套安全机制的通道。
- **HAL 三代演进**：隔离（legacy→HIDL）→ 简化（HIDL→AIDL）。隔离是原则，双栈是代价，归一是终局。

- **分区即解耦**：Treble 用物理分区强制接口稳定，让 system 升级与 vendor 解耦；GSI/GKI 是同一思想的镜像/内核版。
- **APEX 模块化**：把“必须随系统走但又想独立升级”的部分（ART、bionic）打包成可挂载的 mini 分区。
- **system_server 集中 vs 关键服务独立**：集中减少 IPC 与状态同步成本；SurfaceFlinger/audioserver 独立是为崩溃隔离与实时性——边界画在“故障域”上。
- **SystemServiceRegistry 门面**：进程内缓存 Binder 代理，屏蔽服务获取细节，App/系统应用同一入口。
- **属性系统的不对称设计**：读极简（mmap）、写集中（init 仲裁），适配最早期的运行环境。
- **Zygote 预加载 + COW**：以一次加载换千百次 fork 的速度与内存，是 Android 多进程模型成立的前提。
- **SELinux 强制访问控制**：DAC 会 root 绕过，MAC 不允许；策略与代码同树（system/sepolicy），改服务必查策略。

第十六章 深水区专题一：SELinux 策略实战与 APEX 内部

16.1 SELinux 策略编写实战（读改两件套）

策略源码在本树的体量会刷新你的认知：仅 `system/sepolicy/private/` 下就有 **202 个 .te 文件**。编写实践的三板斧：

text 给新服务 mysvc 补策略的标准动作

```
① 定义域：mysvc.te 写 type mysvc, domain; type mysvc_exec, exec_type, file_type;
+ init_daemon_domain(mysvc)
② 声明清单：file_contexts 加 /system/bin/mysvc u:object_r:mysvc_exec:s0
service_contexts 加 mysvc u:object_r:mysvc_service:s0
③ 最小放行：按 avc denied 逐条补 allow——
neverallow 是"宪法"（domain.te:119/:137 起大量出现）：
编译期硬校验，任何 allow 与其冲突直接拒绝构建
④ 应用侧打标：seapp_contexts（sepolicy/private/seapp_contexts）
按 package name/uid 给 App 域（untrusted_app 等）
```

逐段精读：`audit2allow < avc.log` 能自动生成候选 allow，但**永远不要无脑采纳**——它只会"让报错消失"，不知道语义该不该放行。生产实践是先看 neverallow 是否已经替你拒绝了这条路（例如禁止普通域写 shell_data_file），再决定是改架构还是补权限。

16.2 APEX 激活与回滚内部（apexd）

> **源码锚点 · apexd 关键文件**
> system/apex/apexd/apexd_main.cpp:99 main 入口（init 的 post-fs-data 期拉起）
apex_file_repository.cpp：收集 /system/apex 与 /data/apex（会话安装位）
apex_database.cpp：活跃 APEX 挂载表（loop 设备与 /apex 视图）
apexd_session.cpp：staged 会话与激活/回滚状态机

激活流程：开机 apexd 扫描 → 校验 AVB 公钥与版本 → loop 挂载到 `/apex/<name>@<ver>` → 类路径（apex_classpath.cpp）注入 ART。**回滚**：staged 会话默认"试运行"（boot flag），连续两次启动失败自动退回旧版——与会话状态机（session state：staged→activated→reckoned）一体，这就是 Mainline"敢独立发版"的底气。

16.3 Mainline 模块边界速查（哪些已模块化）

模块	内容	升级通道
com.android.art	ART 运行时与 dexopt（书6 §3.3 的 artd）	Google Play 系统更新
com.android.runtime	bionic 与运行时库	同上
com.android.tethering	网络共享与连接模块	同上
com.android.permission	PermissionController（权限 UI/逻辑）	同上
com.android.scheduling	JobScheduler 域逻辑	同上
com.android.wifi	Wi-Fi 栈（部分）	同上
com.android.media / mediaprovider	媒体编解码配置/存储提供者	同上
（对照）framework/system 分区本体	AMS/WMS/PMS 等核心服务	仅整机 OTA

判断口诀：**"策略与 UI"可进模块，"事实与账本"留在系统**（与书6 权限对话框同一条分界线）。

第十七章 深水区专题二：Soong 实战与链接器命名空间

17.1 写一个 Android.bp 的完整心智

text 给书4 的假设服务 mysvc 写构建描述（Soong 语法要点）

```
cc_binary { // 产物：/system/bin/mysvc
name: "mysvc",
srcs: ["mysvc.cpp"],
shared_libs: ["libbinder", "libutils", "libcutils"], // 依赖→dt_needed
init_rc: ["mysvc.rc"], // 随二进制安装 rc（书2 的解析入口！）
vintf_fragments: ["mysvc.xml"], // HAL 时必须：VINTF 清单
```

```
relative_install_path: "hw", // 装到 /system/bin/hw（HAL 惯例）
proprietary: true, // 归入 vendor 分区（对应 soong 配置）
stem: "vendor.mysvc-default", // HAL 服务命名惯例
}
```

变体轴的直觉：Soong 会按 arch（arm64）×os（android/vendor）×link（shared/static）自动展开出多个构建目标——**你声明的是"家族"，构建系统生的是"全家福"**。改 bp 后 `m module-info` 可查模块间依赖图。

17.2 链接器命名空间与 linkerconfig

Android 的动态链接不是"一锅端"：**linker namespace**（配置源 `system/linkerconfig/`）按进程把 so 划入不同命名空间——vendor 进程只能见 vendor 命名空间导出的符号、framework 的 libbinder 与 vndbinder 版本互不可见（Treble 在**链接期**的隔离，比运行期 SELinux 更早）。bionic 与 glibc 的差异清单：缺 NSS/PAM、自带属性/日志 API、ABI 随 NDK 冻结——读码时遇到 `__system_property_get` 这类"陌生 libc 函数"，答案都在 bionic。

17.3 sanitizer 家族（内存安全工具链）

ASAN（插桩+影子内存，构建开关 sanitize: {address: true}）、HWASAN（ARMv8 TBI 硬件标签，内存开销小得多）、**MTE**（内存标签硬件化，书2 forkSystemServer 里 `arm64.memtag` 属性即开它）、GWP-ASan（低概率采样，生产期兜底）。系统服务默认带 GWP-ASan 抽奖（SystemServer.java:763 的 GWP_ASAN_LEVEL_LOTTERY 同源思想）——**内存安全不是单一工具，是分层的"概率×覆盖"组合拳**。

第十八章 边境机制三讲：JNI、Context 家族与 native 智能指针

前三册反复跨过 Java/Native 边境却未教过"过境规则"。本章补上三门必修前置课。

18.1 JNI：两种注册与三类引用

注册：静态（Java 声明 native、库加载时 `RegisterNatives` 显式绑定——框架全部走此路，`AndroidRuntime.cpp` 的 startReg 一次注册几百个）与动态（`JNI_OnLoad` 里按签名查找，JNI 调试器友好但慢）。**引用三态**：局部引用（方法栈内，帧返回即回收——**循环里不释放会撑爆表**）、全局引用（跨线程/跨调用持有，`NewGlobalRef/DeleteGlobalRef` 成对）、弱全局（不阻止 GC，用前必须 `GetObjectLocalRef` 升格检查）。第三册 `javaObjectForIBinder` 里 BinderProxy 的 nativeData 长期持有就是全局引用的现场；`JavaBBinderHolder` 的弱缓存（:462-485）则是弱引用的现场。**异常语义**：JNI 异常不中断 native 执行，必须显式 `ExceptionCheck` 后返回——书3 的 transact JNI 里每一跳都有这个检查。

18.2 Context 家族：一把钥匙配千把锁

Context 是"访问系统环境的句柄"，族谱三支：**ContextImpl**（`core/java/android/app/ContextImpl.java:192`，唯一实现，持有 LoadedApk/权限主题/显示区）——**ContextWrapper**（纯委托壳，Service/Application 都是它的子类）——以及各组件 Context（Activity 的还带窗口与主题注入）。系统侧的"系统 Context"（书2 createSystemContext :882）是给 system_server 里那些"没有界面的应用"配的 ContextImpl。理解两件事就够用：**getSystemService 的缓存挂在 ContextImpl 上**（书1 §5.2 的 fetcher 缓存）；**startActivity 的"令牌"来自 Context 背后的窗口/Activity**——用 Application Context 启动有 UI 依赖的 Activity 会失败，就是令牌缺失的经典报错。

18.3 sp/wp/RefBase：native 世界的 GC

libutils 自带一套引用计数 GC（`system/core/libutils/include/utils/RefBase.h:281`）：**sp<T>** 强指针（构造 incStrong/析构 decStrong，计数归零销毁）、**wp<T>** 弱指针（只升弱计数，-promote() 升格检查对象是否存活——**native 世界的"软引用"**）。为什么不用裸指针：Binder 世界对象跨线程跨进程共享（书3 的 BBinder/BpBinder 全是 RefBase 子类），需要确定性生命周期；为什么不用 std::shared_ptr：历史先于标准库成熟，且弱引用语义可自定义。第三册的 BC_ACQUIRE/BC_INCREFS 命令，本质是**把这套计数跨越进程镜像**——两端各自 sp，驱动保证两本账同步。读懂 RefBase，书3 §5.4 的引用计数命令族就从"名词"变成"机制"。

> **源码锚点 · 本章锚点**
> frameworks/base/core/jni/AndroidRuntime.cpp startReg（静态注册大表）
core/java/android/app/ContextImpl.java:192 / LoadedApk.java:107
system/core/libutils/include/utils/RefBase.h:281
案例：android_util_Binder.cpp:749（全局引用）/:462（弱缓存）

附录 A 版本指纹与阅读路线

版本指纹（`build/make/core/version_defaults.mk`）：`DEFAULT_PLATFORM_VERSION := TP1A`；`PLATFORM_SDK_VERSION := 33`；`PLATFORM_SECURITY_PATCH := 2022-08-05`。即 Android 13 Tiramisu 初始版本。

下一步：第二册《系统启动主线》将从内核拉起 init 的第一行代码开始，沿 `init → Zygote → SystemServer → Launcher` 逐函数走完开机全程；第三册《Binder IPC 全链路剖析》将把本册反复出现的“经 Binder 调用”逐跳展开。三册读完，你面对任何 Android 系统问题，都能定位到“哪一层、哪个进程、哪条调用链”。

附录 B 系统进程归属速查

进程	pid 典型	出生方式	一句话职责
init	1	内核启动	万物之父，rc 解释器与服务监护
ueventd	2	init fork（argv[0] 切换）	uevent 处理、/dev 节点与固件加载
servicemanager	-	init 服务	Binder 服务名录
zygote（主/辅）	-	init 服务	App 与 system_server 之母
system_server	-	zygote fork	160+ 框架服务宿主
surfaceflinger	-	init 服务	图形合成
audioserver	-	init 服务	音频策略与混音
cameraserver	-	init 服务	相机 HAL 宿主
mediaserver	-	init 服务	媒体编解码
netd	-	init 服务	网络配置执行者
vold	-	init 服务（early-fs）	存储卷与加密
lmkd	-	init 服务	低内存查杀
logd	-	init 服务	日志中枢
bootanimation	-	init 服务（core animation）	开机动画
adbd	-	init 服务	调试桥（user 版受限）
installd	-	init 服务	APK 安装/dexopt 执行者
*.vendor HAL 进程	-	init 服务（class hal）	硬件服务（vendor 分区）

附录 C 常用系统属性速查（带源码出处）

属性	含义	设置/读取位置
ro.build.version.sdk	SDK 版本（33）	构建期写入，build.prop
ro.product.cpu.abilist	CPU ABI 列表	决定 zygote 单/双形态
ro.bootmode	启动模式（charger=充电）	init.cpp:978 分支触发链
ro.crypto.state	加密状态（encrypted/unencrypted）	zygote-start 触发条件
sys.boot_completed	开机完成权威标志	AMS.java:5149 设置

属性	含义	设置/读取位置
init.svc.zygote	服务状态机（running/restarting/stopped）	service.cpp:684 NotifyStateChange
persist.sys.timezone	时区	SystemServer.run:777 校正
ctl.start / ctl.stop	服务控制命令	property_service.cpp:185 分流
debug.sf.*	SurfaceFlinger 调参	SF 读取
dalvik.vm.*	ART 参数（heapsize 等）	libart 读取

附录 D 术语表

术语	展开	一句话解释
AOSP	Android Open Source Project	Android 开源项目及其源码树
ART	Android Runtime	5.0 起的运行时，替代 Dalvik
APEX	Android Pony EXpress	可独立升级的模块容器（.apex）
AVB	Android Verified Boot	镜像完整性校验链
CTS	Compatibility Test Suite	兼容性认证测试集
GKI	Generic Kernel Image	通用内核镜像（init_boot 分离配套）
GSI	Generic System Image	可刷的通用 system 镜像
HAL	Hardware Abstraction Layer	硬件抽象层
HIDL	HAL Interface Definition Language	Treble 时代的 HAL IDL（冻结维护）
JIT/AOT	Just/Ahead Of Time	即时/提前编译
LMKD	Low Memory Killer Daemon	用户态低内存查杀
MAC/DAC	Mandatory/Discretionary Access Control	强制/自主访问控制
PMS	PackageManagerService	包管理服务
PSI	Pressure Stall Information	内核压力信息（lmkd 输入）
RC	Android Init Language	.rc 配置文件（init 语言）
SEPolicy	SELinux Policy	强制访问控制策略
Treble	Project Treble	system/vendor 架构解耦工程
USAP	UnSpecialized App Process	Zygote 预孵化进程池
VINTF	Vendor Interface	设备接口清单与兼容矩阵
WMS	WindowManagerService	窗口管理服务

附录 E 上手实验（半天量）

实验一“服务普查”：`adb shell service list` 统计服务总数，找出十个你认识的与十个陌生的；陌生的逐一 `dumpsys` 看自述，猜职责再回源码验证（services/core 对应目录）。

实验二“属性观察”：`adb shell getprop | wc -l` 数属性量级；`getprop init.svc.zygote`、`getprop sys.boot_completed`、`getprop ro.build.version.sdk` 各自对应本册哪一章；用 `setprop` 试试写 `ro.` 前缀，观察拒绝行为（property_service.cpp:176 的只读检查）。

实验三“SELinux 初体验”（userdebug）：`adb shell getenforce` 看模式；`setenforce 0` 前后各跑一次某功能，`logcat -b events | grep avc` 收集差异；把一条 denied 翻译成策略语言（对照第十一章 11.2 的表）。

实验四“目录定向”：不看本册，凭记忆说出 frameworks/native/cmds/servicemanager、system/core/rootdir、frameworks/base/services/core/java/com/android/server/wm 各住着什么；再全书检索验证。

每个实验写三行结论：看到了什么、对应哪条源码锚点、哪里出乎意料。